RosterIQ-HiLabs Hackathon Report

Team Zopita | Pradipta Sundar Sahoo | Dhruv Khandelwal

1. Problem Approach — How We Interpreted the Problem and Scoped Our Solution

Healthcare payers receive thousands of provider roster files monthly from source systems such as AvailityPDM, DPE, PDM, and MedEnroll. Each file passes through a 7-stage processing pipeline:

PRE_PROCESSING → MAPPING_APPROVAL → ISF_GENERATION → DART_GENERATION → DART_REVIEW → DART_UI_VALIDATION → SPS_LOAD.

Files get stuck, fail validation, or degrade market SCS% (Transaction Success Rate), and operations teams have no unified view — they must manually triage across disconnected data sources.

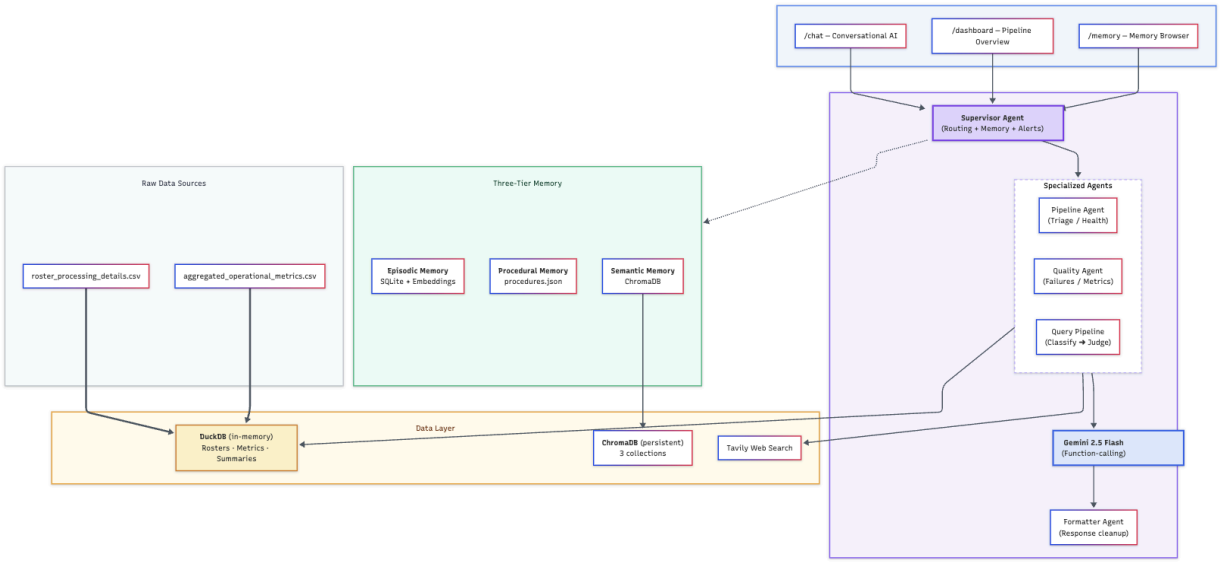
We interpreted the core problem as one of operational blind spots: teams knew something was wrong but lacked the tooling to quickly answer "why" and "what to do next." Rather than build a static dashboard, we scoped our solution as a conversational AI agent — one that could answer natural language questions, run multi-step diagnostic procedures, surface root causes across multiple data dimensions, and remember what it had previously found.

The solution is built on two raw data layers: `roster\_processing\_details.csv` (one row per Roster Operation with per-stage durations, health flags, IS_STUCK, IS_FAILED, failure reasons, and record counts) and `aggregated\_operational\_metrics.csv` (monthly market-level transaction success/failure counts and SCS% per state and client). From these inputs, RosterIQ builds a fully enriched analytical layer at startup with 15+ derived columns, five DuckDB summary tables, and a three-tier memory system so every query builds on prior investigations.

We deliberately scoped out manual ETL, real-time data ingestion, and fine-tuning in favour of a system that could reason deeply over the existing data, self-correct its own SQL, update its own procedural playbooks based on user feedback, and get measurably better across a session.

2. Architecture & Schematics — Diagrams of Agent Memory, Tool Integrations, and Data Flow

System Overview

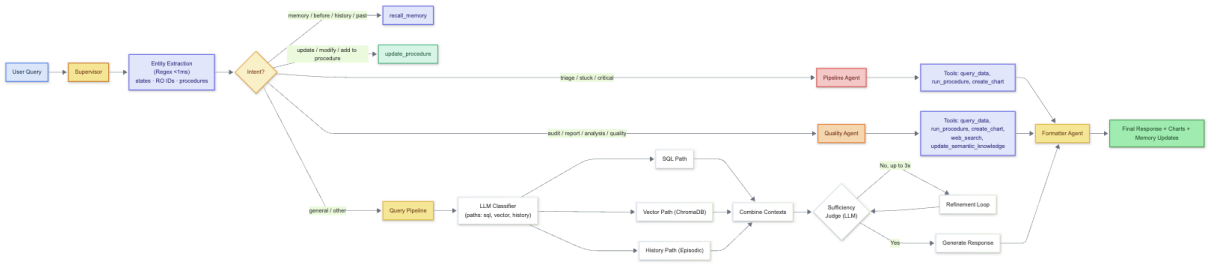


Raw CSVs (`roster\_processing\_details.csv` and `aggregated\_operational\_metrics.csv`) feed into `data\_loader.py` at startup, which produces five enriched DuckDB tables: `roster`, `metrics`, `state\_summary`, `org\_summary`, and `stage\_health\_summary`. A schema cache is built from these tables and dynamically injected into every LLM prompt.

The backend runs on FastAPI + Uvicorn (port 8000). Every incoming message hits the Supervisor Agent, which performs regex entity extraction (<1 ms) to detect state codes, RO IDs, LOBs, and procedure names, then routes by intent to one of three specialized agents: the Pipeline Agent (triage, health), the Quality Agent (failures, metrics, web search), or the Query Pipeline (general multi-path retrieval). All agents call Gemini 2.5 Flash via function-calling and their responses are cleaned up by a Formatter Agent before returning to the frontend.

The frontend runs on Next.js 16 + React 19 (port 3000) across three pages: `/chat` (conversational AI with tool call cards and procedure learning cards), `/dashboard` (KPI cards, charts, proactive alerts), and `/memory` (browser for all three memory tiers). Deployment is via Docker Compose with two containers.

Multi-Agent Routing



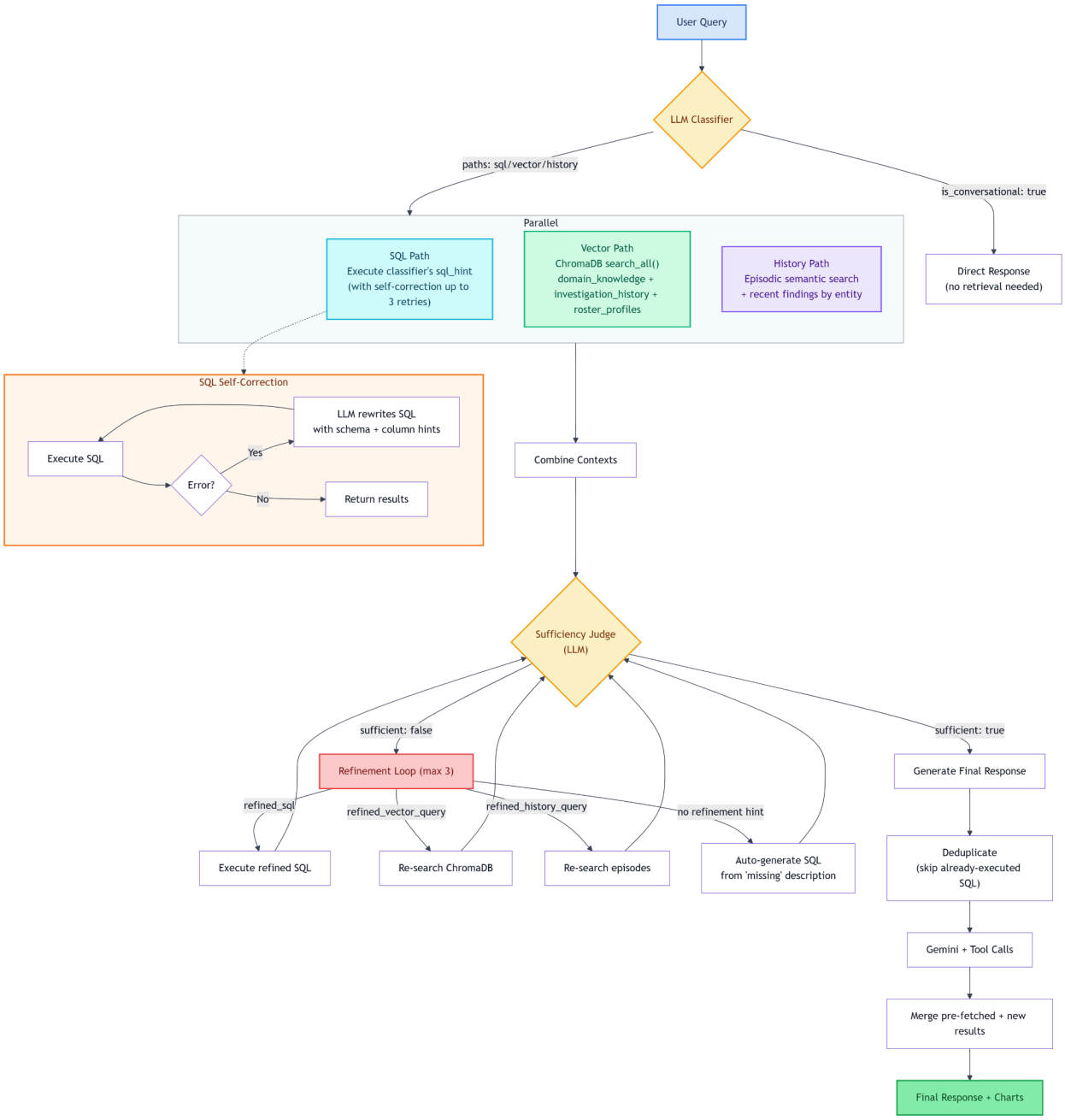
User Query → Supervisor → Regex Entity Extraction → Intent Classification → one of:

- Pipeline Agent (triage / stuck / health) with tools: `query\_data`, `run\_procedure`, `create\_chart`
- Quality Agent (audit / quality / market / retry) with tools: `query\_data`, `run\_procedure`, `create\_chart`, `web\_search`, `update\_semantic\_knowledge`
- `recall\_memory` (memory / before / history queries) — episodic semantic search
- `update\_procedure` (update / modify procedure) — version bump + diff
- Query Pipeline (general) — classify → parallel retrieval → judge → generate

All routes pass through the Formatter Agent before final output.

Multi-Path Query Pipeline (for general queries)

User Query → LLM Classifier. If conversational, return Direct Response. Otherwise, run three retrieval paths in parallel:

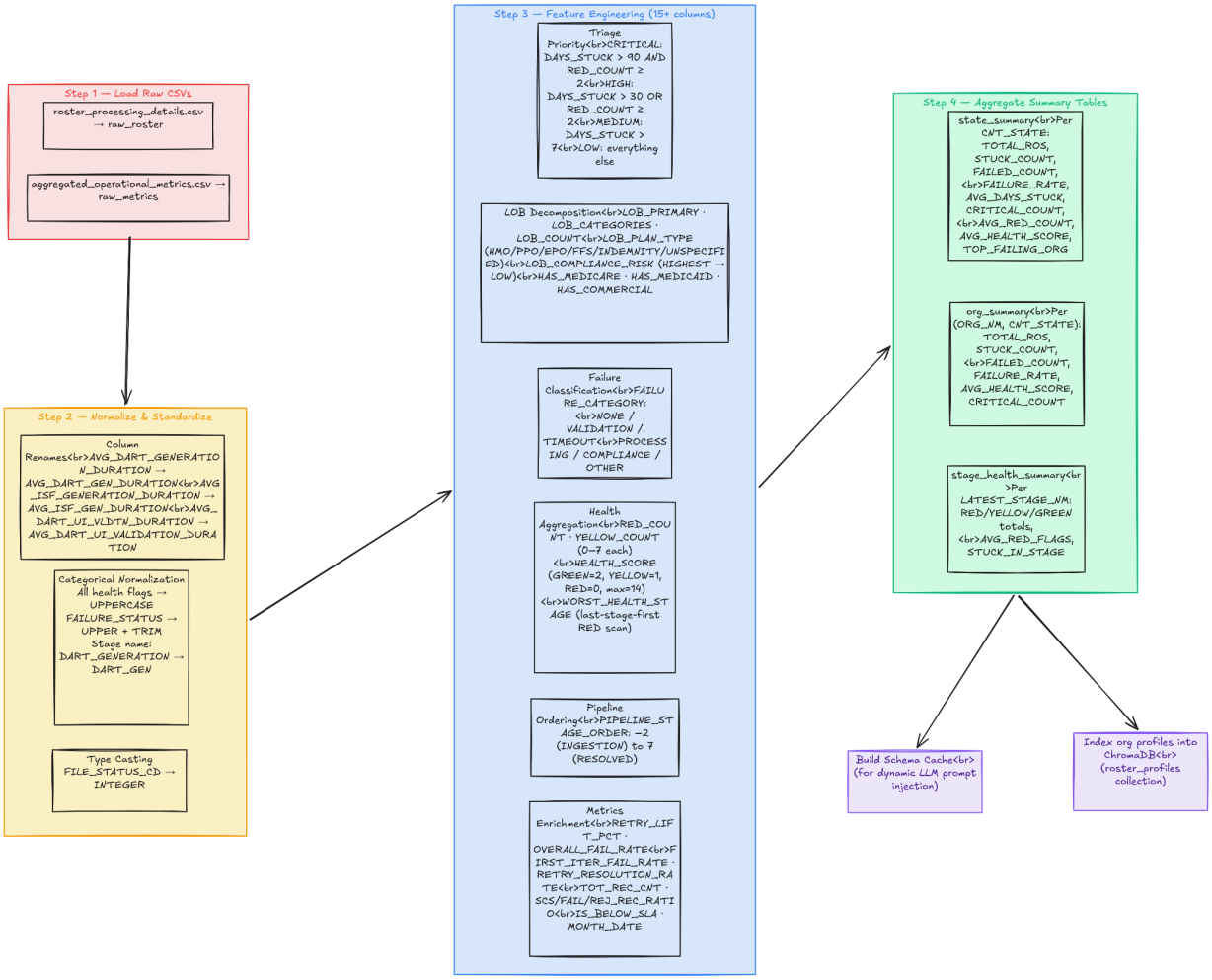


- **SQL Path**: execute `sql\_hint`, self-correct up to 3 times on error.
- **Vector Path**: ChromaDB search across all 3 collections.
- **History Path**: episodic semantic search + recent findings.

Results are merged and passed to a Sufficiency Judge (up to 3 refinement loops). If sufficient, generate a final response via Gemini. If insufficient, refine queries and loop. Final output deduplicates already-executed SQL, merges results, and returns response with charts.

Empirical sufficiency rates: ~70% after pass 1, ~90% after pass 2, ~97% after pass 3.

Data Preprocessing Flow

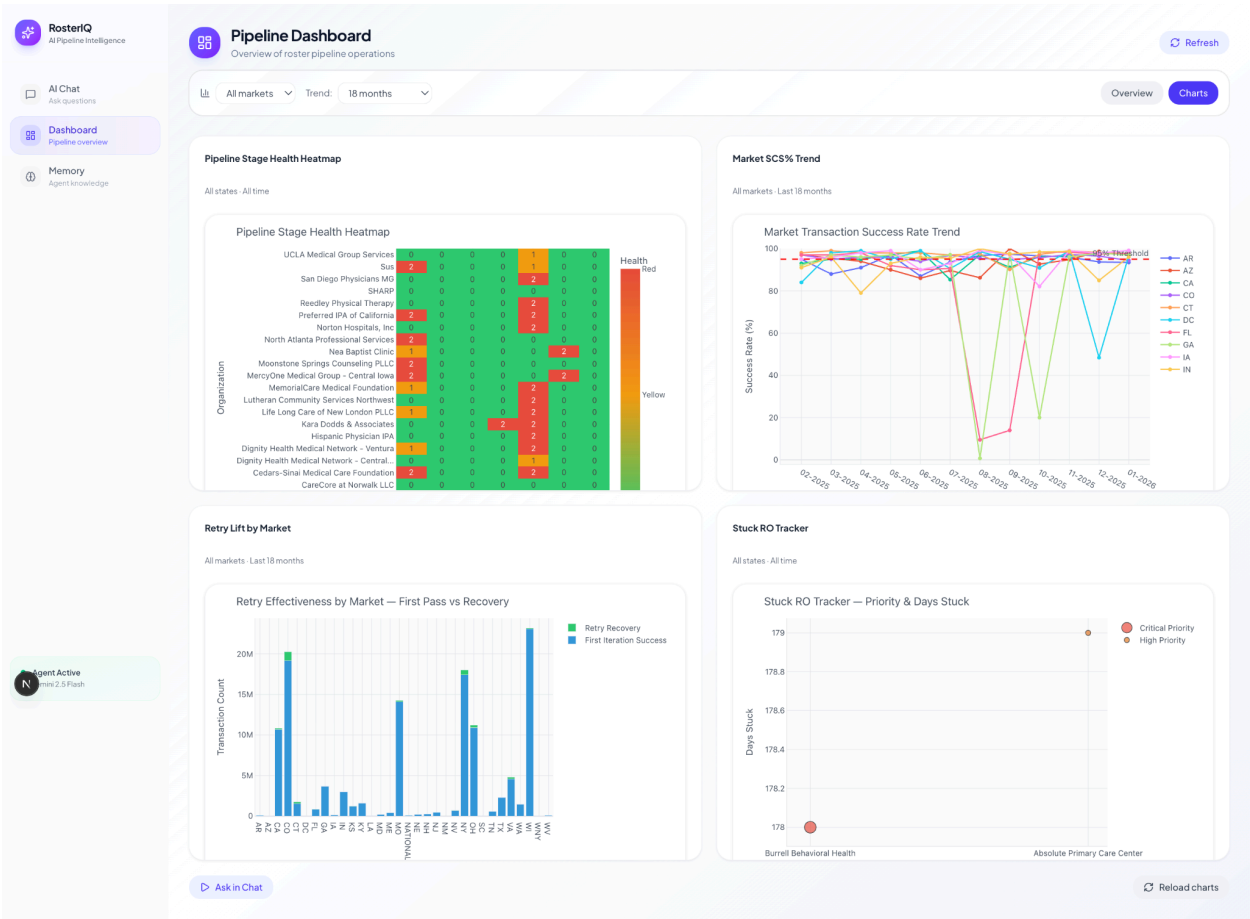


1. **Step 1** — Load raw CSVs into DuckDB.
2. **Step 2** — Normalize: health flags to UPPERCASE, FAILURE_STATUS trimmed, DART_GENERATION renamed DART_GEN, FILE_STATUS_CD cast to INTEGER.
3. **Step 3** — Feature engineering:
    - PRIORITY (CRITICAL/HIGH/MEDIUM/LOW based on DAYS_STUCK and RED_COUNT thresholds)
    - LOB fields (LOB_PRIMARY, LOB_PLAN_TYPE, LOB_COMPLIANCE_RISK, HAS_MEDICARE, HAS_MEDICAID, HAS_COMMERCIAL)
    - FAILURE_CATEGORY (NONE/VALIDATION/TIMEOUT/PROCESSING/COMPLIANCE/OTHER)
    - RED_COUNT and YELLOW_COUNT (0–7)

- HEALTH_SCORE (GREEN=2, YELLOW=1, RED=0, max=14)
- WORST_HEALTH_STAGE, PIPELINE_STAGE_ORDER
- Metrics-side columns (RETRY_LIFT_PCT, RETRY_RESOLUTION_RATE, SCS_REC_RATIO, FAIL_REC_RATIO, IS_BELOW_SLA, MONTH_DATE as parsed timestamp).

4. **Step 4** — Build aggregate summary tables: `state\_summary`, `org\_summary`, `stage\_health\_summary`. Step 4 also populates the schema cache and ChromaDB org profiles.

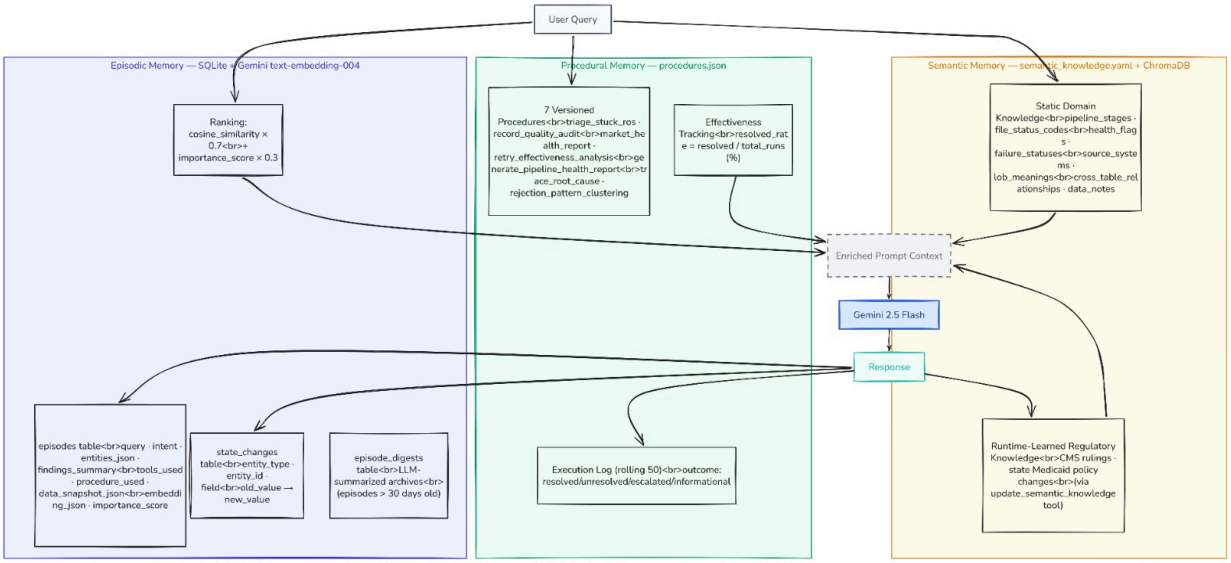
Visualization Engine (4 Plotly chart types)



- **`health\_heatmap`**: Org × 7 pipeline stages, colored GREEN/YELLOW/RED. Source: top 30 orgs by health score.
- **`stuck\_tracker`**: Scatter of stuck ROs by org × DAYS_STUCK, sized by RED_COUNT, colored by PRIORITY.
- **`market\_trend`**: Monthly SCS% lines per market with 95% SLA reference line.
- **`retry\_lift`**: Stacked bar of First Iteration Success + Retry Recovery per market.

3. Memory Design Decisions — How and Why We Implemented Each Memory Type

We implemented a three-tier memory system reflecting how human analysts actually retain and apply knowledge: they remember what they investigated (episodic), what procedures worked (procedural), and what domain terms mean (semantic). A single undifferentiated memory store would collapse these into noise.



Episodic Memory — "What did we find before?" (SQLite + Gemini text-embedding-004)

We store every query and its findings as an episode with fields: `query`, `intent`, `entities\_json`, `findings\_summary`, `tools\_used`, `procedure\_used`, `data\_snapshot\_json`, `embedding\_json`, and `importance\_score`. Episodes are embedded using Gemini text-embedding-004 and retrieved by a weighted ranking formula: `cosine\_similarity × 0.7 + importance\_score × 0.3`.

Why SQLite rather than ChromaDB alone? Episodic memory requires both semantic search (find similar past queries) and relational queries (filter by session, count episodes for consolidation triggers, order by timestamp). ChromaDB cannot do relational operations, so we use SQLite as the primary store and compute cosine similarity in Python at retrieval time.

Importance scoring is computed automatically: web search or procedure execution adds +0.2; chart generation adds +0.15; mentions of CRITICAL, STUCK, or RED in findings add +0.1. This ensures high-stakes investigations surface first in future retrievals.

Each episode also stores a full pipeline data snapshot (stuck ROs by state, SCS% by market). On session start, `generate\_session\_briefing()` diffs the current snapshot against the previous one and surfaces changes automatically — for example, "TN stuck ROs: 5 → 2 since last session."

Consolidation: after 100+ episodes, entries older than 30 days are LLM-summarized into `episode\_digests` to keep retrieval fast without losing historical signal.

Procedural Memory — "What workflow should I follow?" (`procedures.json`, versioned)

We store 7 named diagnostic procedures: `triage\_stuck\_ros`, `record\_quality\_audit`, `market\_health\_report`, `retry\_effectiveness\_analysis`, `generate\_pipeline\_health\_report`, `trace\_root\_cause`, and `rejection\_pattern\_clustering`. Each procedure has a version number, steps, parameters, modification_history, and a rolling execution log (last 50 runs) with outcomes categorized as resolved, unresolved, escalated, or informational.

The key design decision was to make procedures user-editable at runtime. When a user says "also include SKIP_REC_CNT in the quality audit," the `update\_procedure` tool adds the step, bumps the version, records the diff, and the `/chat` UI renders a `ProcedureLearningCard` showing exactly what changed and why. This makes agent self-improvement transparent and auditable rather than opaque.

Effectiveness scores (`resolved\_rate = resolved\_count / total\_runs`) are injected into the LLM prompt so it recommends procedures that have historically worked for a given type of problem, not just procedures that exist.

Semantic Memory — "What does this term mean?" (`semantic\_knowledge.yaml` + ChromaDB)

Static domain knowledge is injected into every LLM prompt: pipeline stage descriptions, health flag meanings, LOB compliance hierarchies, and FILE_STATUS_CD operational meanings. The model never has to guess what a status code means.

Two distinctions are explicitly encoded that LLMs consistently confuse:

- `FAIL\_REC\_CNT` (processing failure — a pipeline error) vs `REJ\_REC\_CNT` (validation/compliance rejection — non-conformant source data). These are different root causes requiring different responses.
- Similarly, MONTH is stored as MM-YYYY strings whose lexicographic sort is wrong; all temporal queries are explicitly routed to use MONTH_DATE (the parsed TIMESTAMP column).

We also store cross-table relationships explicitly: how `roster.CNT\_STATE` maps to `metrics.MARKET`, what RUN_NO > 1 means, and why the two tables cannot be naively joined. This prevents hallucinated joins.

Semantic memory is also runtime-extensible: the Quality Agent can call `update\_semantic\_knowledge` to persist new regulatory knowledge (e.g., CMS rulings, state Medicaid policy changes from Tavily web search) into both the YAML and ChromaDB. Runtime-added entries are visually flagged in the `/memory` browser with their modification history and reason.

We use three separate ChromaDB collections — `domain\_knowledge`, `investigation\_history`, and `roster\_profiles` — rather than one. Mixing domain definitions, investigation logs, and org profiles in a single collection degrades retrieval precision because all three have very different semantic profiles.

4. Innovations — Novel Techniques, Design Patterns, and Creative Extensions

Geometric Mean Driver Scoring for Root Cause Attribution

Standard root cause approaches rank by failure rate alone, which over-surfaces low-volume outliers. Our `driver\_score = √(failure\_rate × share\_of\_failures)` is a geometric mean that requires both high failure rate and high share of total failures to rank highly. A source system with 100% failure rate but 0.1% volume share scores far lower than one with 30% failure rate and 30% share — the latter is far more actionable for operations teams. This formula is applied identically to both source systems (SRC_SYS) and lines of business (LOB_PRIMARY), giving a unified ranking across dimensions.

10-Dimension Statistical Root Cause Chain

The `trace\_root\_cause` procedure runs `cross\_table\_state\_analysis()` across 10 correlated dimensions in a single pass:

- Market SCS trend
- Baseline failure stats
- Stage blame scores (lift = relative over-representation of RED flags among failed ROs)
- Source system driver scores
- LOB driver scores
- Retry pattern impact vs global average
- Cross-dimension hotspots at (SRC_SYS × LATEST_STAGE_NM) intersections
- A ranked unified driver list
- SCS decline context with directional acceleration
- A synthesized causal chain narrative ordered by severity

Regex-First Entity Extraction

Rather than burning an LLM call on entity extraction, we use deterministic regex for the bounded entity space: 50 state codes, RO-\d+ patterns, 7 procedure names, and known LOB values. This gives 100% precision with <1 ms latency and zero hallucination risk. The LLM is only invoked once the entities are already known.

SQL Self-Correction with Schema-Aware Fuzzy Matching

Blind SQL retries repeat the same errors. Our self-correction loop injects `schema\_provider.find\_column\_corrections()` output into the retry prompt — this performs fuzzy matching on the failed column name and returns the correct column, data type, and sample values. The most common failure classes caught are:

- Misspelled column names
- `=TRUE` vs `=1` boolean comparisons
- Case mismatches in categorical values (e.g., 'red' vs 'RED')
- Full state names instead of abbreviations

This brings SQL first-attempt success from ~60% (with inline LLM derivations) to ~90%, and effective accuracy to ~98% with correction.

Importance-Weighted Episodic Retrieval

Rather than pure cosine similarity for memory retrieval, we use a composite ranking: `cosine\_similarity × 0.7 + importance\_score × 0.3`. Importance is computed automatically from what happened during the episode (web search, chart generation, critical-severity mentions). This means a highly similar but low-stakes past query does not crowd out a less-similar but critical investigation.

Session Briefing with Delta Detection

On every session start, the system diffs the current pipeline snapshot against the previous session's snapshot and generates a natural language briefing: which markets improved, which worsened, which stuck ROs were resolved. This is not a standard chat feature — it mirrors how a human analyst would want to start their shift.

Versioned Procedural Self-Improvement

The agent can modify its own diagnostic playbooks at runtime in response to user feedback. Procedure version history, step diffs, and execution outcomes are all stored and surfaced in the UI, making the agent's self-improvement auditable. This is meaningfully different from a static prompt — the playbooks genuinely accumulate operational knowledge across sessions.

Pre-Computation as LLM Reliability Architecture

A deliberate architectural choice: all 15+ derived columns (PRIORITY, FAILURE_CATEGORY, HEALTH_SCORE, WORST_HEALTH_STAGE, RETRY_LIFT_PCT, etc.) are computed once at startup rather than derived on-the-fly in SQL. This reduces LLM-generated SQL to simple lookups and WHERE clauses, cutting failure rates from 40–60% (inline derivations) to ~2% with self-correction. The data layer is designed around LLM cognitive limits, not database elegance.

5. Challenges & Learnings — What Didn't Work, What We'd Do Differently, and What Surprised Us

What Didn't Work

LLM SQL Generation Is Unreliable Without Guardrails

Our earliest prototype had the LLM generate SQL with inline derived columns (e.g., `CASE WHEN IS\_STUCK=1 AND DAYS\_STUCK > 7 THEN 'CRITICAL' ...`). Failure rates were 40–60%. The fix — pre-computing everything into named columns at table creation time — seems obvious in retrospect but took real debugging to diagnose. The lesson: design the schema around what the LLM can reliably generate, not what a human engineer would write.

Blind SQL Retries Are Useless

Simply retrying a failed SQL query with the same prompt repeats the same mistake. We found that injecting targeted corrections (the correct column name, its data type, sample values) into the retry prompt reduced errors dramatically. Schema-aware fuzzy matching in the correction step was the key unlock.

Temporal String Sorting Caused Silent Bugs

MONTH stored as MM-YYYY strings sorted incorrectly — 01-2024 came before 12-2023 alphabetically. Every chart that showed trends was silently wrong until we added MONTH_DATE as a parsed timestamp and enforced its use for all ordering. The bug was subtle because the charts still rendered; they just showed the wrong order. This reinforced the value of encoding cross-table relationships and data quirks explicitly in semantic memory rather than relying on the LLM to infer them.

Single-Agent Monolith Failed at Scale

An early version used a single agent with all tools, the full schema, and all semantic context in one prompt. At ~8000 tokens, the LLM routinely ignored parts of the prompt and confused tools. Splitting into focused sub-agents with ~2000-token prompts each resolved this. The tradeoff is routing complexity, but the deterministic regex router made that manageable.

ChromaDB Alone Was Insufficient for Episodic Memory

We initially stored episodic memory in ChromaDB. The problem was that episodic queries often have a relational component: "how many times have we investigated TX?", "what did we find in the last session?", "consolidate episodes older than 30 days." ChromaDB has no relational operations. We migrated to SQLite with in-Python cosine similarity, which handled both needs cleanly.

Sufficiency Checking Was Harder Than Expected

We assumed most queries would be answerable in a single retrieval pass. In practice, multi-faceted questions like "Why is TX SCS% dropping?" genuinely require trend data, failure breakdowns, episodic history, and sometimes regulatory context — none of which any single retrieval path returns completely. The three-pass sufficiency judge with targeted refinement was necessary, not gold-plating. We were surprised that ~30% of non-trivial queries required at least two passes.

What We'd Do Differently

- We would also invest in a more systematic evaluation harness from the start rather than relying on manual spot-checks; debugging SQL failure modes would have been faster with logged query-correction pairs.
- Finally, we would separate the sufficiency judge's refinement logic more cleanly — it currently conflates "insufficient data" with "ambiguous query," which occasionally triggers unnecessary extra retrieval passes on well-scoped questions.

What Surprised Us

- The geometric mean driver scoring formula had a larger qualitative impact on root cause quality than expected — without it, low-volume high-failure-rate systems dominated recommendations even when they were operationally irrelevant.
- The importance-weighted episodic retrieval also proved more valuable than anticipated: pure cosine similarity frequently surfaced unhelpful near-duplicate past queries rather than the most informative prior investigations.